LibreAssistant: Phased Engineering and Implementation Roadmap

Introduction

This document presents the canonical engineering roadmap for the LibreAssistant project. It serves as the direct, actionable translation of the principles and specifications articulated in the *LibreAssistant: Technical Specification and Architectural Framework* into a granular, phased implementation plan. The primary objective of this roadmap is to provide a clear and unambiguous sequence of atomic tasks for developers, project managers, and community contributors, ensuring the final system is a faithful, robust, and secure realization of its founding vision.

The development process is structured into four distinct phases, each with a specific strategic purpose. This phased approach is deliberate, prioritizing the establishment of a robust foundation of governance, legal clarity, and architectural integrity before proceeding to application feature development. This methodology directly embodies the core principle of the "Architect's Education" outlined in the technical specification, which posits that the architects of a system—its developers and leaders—must possess a holistic understanding of its ethical, legal, and design underpinnings. By building the project's "constitution" before its "structure," this roadmap ensures the long-term philosophical and technical durability (*firmitas*) of LibreAssistant.

Phase 1: Project Foundation and Governance ("The Groundbreaking")

This initial phase is the most critical for establishing the project's long-term viability, trustworthiness, and capacity for healthy community growth. It focuses on translating the abstract principles of the "Architect's Oath" and the project's FOSS philosophy into tangible, version-controlled artifacts and automated enforcement mechanisms. Every task in this phase is designed to build the project's social and technical foundation before a single line of application code is written, creating the necessary conditions for a successful, collaborative open-source endeavor.

1.1 GitHub Repository Initialization and Legal Foundation

The first set of tasks establishes the project's public home and its legal framework, fulfilling the foundational commitments to transparency and open-source principles.

Task 1.1.1: Create libre-assistant Public Repository

- **Context:** The project's unwavering commitment to Free and Open-Source Software (FOSS) and radical transparency begins with its physical location and accessibility. A public repository is the non-negotiable prerequisite for community engagement,

auditability, and the practical exercise of the Four Essential FOSS Freedoms: the freedom to run, study, redistribute, and modify the software. This act makes the source code accessible, which is a precondition for Freedoms 1 and 3.
- **Implementation:** A new public repository will be created on the GitHub platform under the name libre-assistant. It will be initialized with a README.md file that provides a concise, high-level summary of the project's purpose: to create an open, private, and user-centric AI assistant.

Task 1.1.2: Add LICENSE.txt with MIT License

- **Context:** This task directly implements Specifications 11.1 and 11.2, which formally declare the MIT License as the project's legal framework. The choice of a permissive license like MIT is a strategic decision justified by its simplicity and its capacity to "encourage the widest possible adoption and contribution". This aligns with the FOSS philosophy by placing minimal restrictions on reuse, fostering a larger and more diverse ecosystem of both contributors and users.
- **Implementation:** A file named LICENSE.txt will be created in the root directory of the repository. This file must contain the full, unmodified text of the MIT License as specified in Specification 11.2. In accordance with Specification 11.3, the copyright holder line must be explicitly set to "The LibreAssistant Project Contributors" to reflect the collective ownership model of the project.

Task 1.1.3: Add Standard Python .gitignore File

- **Context:** This is a foundational housekeeping task essential for maintaining a clean and professional version control history. By preventing common development artifacts, temporary files, and sensitive environment configurations from being committed, it contributes directly to the project's long-term maintainability and structural integrity, a practical application of the principle of *firmitas* (durability).
- **Implementation:** A standard Python .gitignore template will be added to the repository's root directory. This file will be configured to ignore common directories and files such as __pycache__/, .pyc files, virtual environment directories (e.g., venv/, .venv/), and environment variable files (e.g., .env).

1.2 Codification of Foundational Principles and Community Governance

This section translates the project's philosophy into binding, public documents. These are not mere guidelines but are the foundational law of the project, establishing the social contract with users and contributors.

Task 1.2.1: Create CONSTITUTION.md ("The Architect's Oath")

- **Context:** This task is the direct and most critical implementation of Specification 1.1, which mandates the creation of a public, version-controlled "Architect's Oath". This document is explicitly defined not as a mission statement but as the project's "supreme charter" and an "active governance mechanism". Every future feature, contribution, and

community decision can and will be measured against its tenets, providing a clear framework for mitigating "ethical drift" over time.

- **Implementation:** A file named CONSTITUTION.md will be created in the repository root. It will be populated with a clear and unambiguous articulation of the project's core, non-negotiable commitments as defined in Part I of the technical specification:
    - The Four Essential Freedoms of FOSS (Specification 1.2).
    - The principles of Fairness, Accountability, and Transparency (FAccT) in all AI components (Specification 1.3).
    - Unyielding user privacy and data autonomy, using GDPR as the baseline standard and mandating Privacy by Design (Specification 1.4).
    - A commitment to holistic, human-centered design that avoids manipulative patterns (Specification 1.1).

Task 1.2.2: Create CODE_OF_CONDUCT.md

- **Context:** This task fulfills Specification 5.3, which requires a "robust Code of Conduct" to serve as the "foundational 'law'" of the project's community spaces. A well-defined and strictly enforced Code of Conduct is the primary tool for building a "welcoming," "healthy, inclusive, and productive" community. The formal codification of these social rules is a prerequisite for building a diverse and effective contributor base. A clear, enforceable Code of Conduct acts as a trust-building mechanism that lowers the perceived risk for potential contributors, especially those from underrepresented groups, to join the project. This directly impacts the project's long-term health and its ability to attract the cross-disciplinary talent required for the Guilds mandated in Specification 1.5.
- **Implementation:** The project will adopt the **Contributor Covenant** (the latest version available at the time of implementation) as the base template. This is the most widely adopted code of conduct in the open-source world, used by over 100,000 projects including Linux, Google, and Microsoft, and its adoption signals immediate alignment with established and respected community norms. The resulting CODE_OF_CONDUCT.md file must be customized to:
    1. Define a private and secure reporting path for incidents (e.g., an email alias like conduct@libreassistant.org).
    2. Name the initial enforcement body (e.g., "The LibreAssistant Core Maintainers").
    3. Include a clear, transparent enforcement ladder to manage expectations and ensure fairness, as detailed in the table below. This ladder is synthesized from best practices recommended by the OpenSSF and other open-source guides.

Level	Community Impact	Consequence
1. Correction	Use of inappropriate language or other behavior deemed unprofessional or unwelcome in the community.	Private, written warning from community leaders, providing clarity on the violation and an explanation of why the behavior was inappropriate. A public apology may be requested.
2. Warning	A violation through a single incident or a series of actions.	A warning with consequences for continued behavior. No interaction with the people involved, including unsolicited

Level	Community Impact	Consequence
		interaction with those enforcing the Code of Conduct, for a specified period of time.
3. Temporary Ban	A serious violation of community standards, including sustained inappropriate behavior or harassment.	A temporary ban (e.g., 1, 7, or 30 days) from any sort of interaction or public communication with the community for a specified period of time.
4. Permanent Ban	Demonstrating a pattern of violation of community standards, including sustained inappropriate behavior, harassment of an individual, or aggression toward or disparagement of classes of individuals.	A permanent ban from any sort of public interaction within the project community.

Task 1.2.3: Create CONTRIBUTING.md

- **Context:** A clear and accessible contribution guide is essential for lowering the barrier to entry for new contributors, which is a key driver of a healthy and sustainable FOSS ecosystem. This document will explain the practical application of the project's governance and development structures, making it easier for anyone to participate.
- **Implementation:** A CONTRIBUTING.md file will be created in the repository root. The initial version of this document must clearly explain:
    - The project's branching strategy, detailing the roles of the main branch (stable releases) and the develop branch (integration).
    - The standard development workflow: fork the repository, create a feature branch from develop, write code and tests, and submit a Pull Request back to the develop branch.
    - The role of the cross-disciplinary "Guilds" in the review process, as mandated by Specification 1.5. It will state that certain Pull Requests, depending on their scope, will require formal sign-off from designated guilds (e.g., a UI change requires UX Guild review; a data handling change requires Data Governance Guild review).
    - A placeholder section on how to set up the local development environment, which will be updated in Phase 2 to point to the "One-Click" Contributor Environment.

1.3 Establishment of Branching Strategy and Project Management Framework

These tasks establish the technical and procedural guardrails for development, ensuring a stable, transparent, and well-managed workflow.

Task 1.3.1: Configure Branch Protection Rules

- **Context:** This is a direct technical enforcement of the project's development strategy,

ensuring the stability and integrity of the main branch, which must always represent production-ready code. This is a crucial implementation of the *firmitas* principle, preventing accidental or un-reviewed changes from destabilizing the project's core.

- **Implementation:** Within the GitHub repository settings, the main branch will be protected. The protection rules will be configured to:
    - Require a pull request before merging.
    - Require at least one approval from a code owner or core maintainer.
    - Require all status checks (i.e., the CI pipeline jobs) to pass before merging.
    - Disallow force pushes to prevent rewriting of the public history.

Task 1.3.2: Create develop Branch

- **Implementation:** The develop branch will be created from the main branch. It will be set as the default branch for the repository, meaning all new pull requests will target it by default. This branch will serve as the primary integration point for all new features and bug fixes.

Task 1.3.3: Configure GitHub Issues Templates

- **Context:** Standardized issue templates are a key project management tool that streamlines the process of reporting bugs and requesting features. They ensure that all necessary information is provided upfront, reducing back-and-forth communication. Critically, these templates will be designed to explicitly integrate the Guild review process into the project's workflow from the very beginning.
- **Implementation:** A .github/ISSUE_TEMPLATE/ directory will be created. Within this directory, YAML-configured templates will be added for:
    - **Bug Report:** Will include fields for a clear title, reproduction steps, expected behavior, actual behavior, and environment details (e.g., browser, OS, app version).
    - **Feature Request:** Will include fields for a user story ("As a [user type], I want to [action] so that [benefit]"), clear acceptance criteria, and a mandatory checklist section titled "Required Guild Reviews." This checklist will list all potential guilds (e.g., [ ] Ethics Guild, [ ] UX Guild, [ ] Security Guild, [ ] Data Governance Guild), forcing the issue creator to consider the cross-disciplinary implications of their proposal from the outset, in alignment with Specification 1.5.

Task 1.3.4: Set Up GitHub Projects Board

- **Implementation:** A new GitHub Project (board view) will be created and titled "LibreAssistant Master Roadmap." This board will provide the transparent, high-level overview of the project's evolution mentioned in the algorithm. The board will be configured with the following columns to track the lifecycle of an issue: Backlog, To Do (Ready for Development), In Progress, Awaiting Guild Review, Awaiting Final Review, and Done.

1.4 Configuration of the Initial CI/CD Pipeline for Governance Automation

This final set of tasks in Phase 1 establishes the principle of "Automated and Transparent Construction" (Specification 10.1) from day one. The Continuous Integration (CI) pipeline acts as the project's first automated quality and governance gatekeeper.

Task 1.4.1: Create Initial CI Workflow File

- **Context:** The CI/CD pipeline is the "automated assembly line" that enforces quality and consistency across all contributions, as described in Specification 10.3. By automating checks, it ensures that every contribution, regardless of its author, is held to the same high standard.
- **Implementation:** A .github/workflows/ directory will be created, containing an initial workflow file named ci.yml. This workflow will be configured using GitHub Actions to trigger automatically on every pull request that targets the develop branch.

Task 1.4.2: Implement Linter for Markdown Files

- **Implementation:** A job will be added to the ci.yml workflow that uses a standard, publicly available action (e.g., markdownlint-cli) to lint all Markdown files (.md) in the repository. This includes the README.md, CONSTITUTION.md, CODE_OF_CONDUCT.md, and CONTRIBUTING.md. This automated check ensures that all core governance documents and future documentation remain clean, readable, and consistently formatted.

Task 1.4.3: Implement License Header Check

- **Context:** This task provides automated enforcement for the project's best practice, established in Specification 11.3, of including a license header in all source files. This practice ensures maximum legal clarity for any downstream users or developers who may interact with individual files from the project, reinforcing the project's commitment to FOSS principles.
- **Implementation:** A job will be added to the ci.yml workflow that runs a script or a dedicated linter action to verify that all source files (initially just .md files, but configured to expand to .py, .js, .css, etc., as they are added) contain the required copyright and MIT license header text. A pull request that adds a new file without the correct header will fail this check and be blocked from merging.

Phase 2: The Microkernel and Core API ("The Structural Frame")

With the project's governance and legal foundations firmly in place, Phase 2 focuses on constructing the stable, testable, and minimal technical core of the application. This phase is a direct implementation of the *modular or microkernel architecture* detailed in Specification 4.1, which prioritizes achieving extreme durability (*firmitas*) in the central core to enable flexible and safe community-driven feature development (*utilitas*) in subsequent phases. Every task is geared towards building a resilient structural frame upon which the rest of the application will be built.

2.1 Microkernel Core Development

This section details the creation of the application's central nervous system, using the mandated FOSS technology stack.

Task 2.1.1: Scaffold the Application using the FOSS Tech Stack

- **Context:** This task brings to life the technology choices mandated in Specification 2.1 and outlined in Table 1 of the technical specification. The specification is unequivocal: the core stack must consist "exclusively of components governed by recognized Free and Open-Source Software licenses" to prevent vendor lock-in and ensure maximum transparency.
- **Implementation:** A new Python project will be initialized. Following the specification, FastAPI will be used as the web framework. Its modern, high-performance, and asynchronous capabilities make it an ideal choice for building the efficient and robust APIs required for the assistant's microkernel architecture. The initial project directory structure will be established (e.g., a /libre_assistant source directory, a /tests directory, and a /docs directory) to promote a clean and logical organization from the outset.

Task 2.1.2: Develop the Minimal Microkernel Services

- **Context:** This is the primary implementation of Specification 4.1, which calls for building a "stable, minimal central 'core'" responsible for only the most essential services. The goal is to keep the core as small and simple as possible to maximize its stability and testability.
- **Implementation:** The initial, minimal stubs for the three essential microkernel services will be developed:
    1. **Main Application Loop:** The primary entry point of the application, responsible for initialization and startup.
    2. **User State Management:** A basic service to handle user session data. Initially, this can be implemented in-memory for simplicity, with the clear architectural intention to connect it to the PostgreSQL database service later.
    3. **Plugin API Endpoint:** A single, stable API endpoint (e.g., /api/v1/invoke) will be created. This endpoint is the heart of the microkernel's public interface. It will be designed to receive requests, identify the target plugin, and dispatch the request accordingly. This endpoint defines the immutable API contract that all future plugin modules must adhere to.

2.2 Implementation of Test-Driven Development (TDD) and Quality Gates

This section focuses on embedding quality into the development process itself, making durability an outcome of the process, not an afterthought.

Task 2.2.1: Write Unit Tests for All Microkernel Components

- **Context:** This task is a direct embodiment of the *firmitas* principle. The technical specification demands that the core be "rigorously tested and maintained for maximum

durability". Adopting a Test-Driven Development (TDD) approach, where tests are written alongside or even before the application code, is the most effective methodology for achieving this rigor.
- **Implementation:** For every function, class, and API endpoint created in Task 2.1.2, corresponding unit tests will be written using the pytest framework. The test suite for the microkernel core must achieve a code coverage of over 90%, ensuring that almost every line of this critical code is verified by an automated test.

Task 2.2.2: Integrate Test Suite into CI Pipeline

- **Context:** This task automates the enforcement of the quality standards established in the previous task. As mandated by Specification 10.3, the CI system must run a "comprehensive suite of automated tests" on every pull request, acting as an impartial gatekeeper.
- **Implementation:** The .github/workflows/ci.yml file created in Phase 1 will be updated. A new job will be added to install Python dependencies and run the full pytest suite on every push to a pull request. The branch protection rules for main and develop will be updated to require this new "test" job to pass before any merge can occur. This creates a non-negotiable quality gate in the development workflow.

2.3 Creation of the "One-Click" Contributor Environment

This section addresses a critical factor in open-source project success: lowering the barrier to contribution.

Task 2.3.1: Create docker-compose.yml and Dockerfiles

- **Context:** This task directly implements Specification 10.2, which calls for a "fully automated way to set up a complete local development environment". This is not merely a developer convenience; it is a critical dependency for the effective functioning of the entire Guild-based governance model established in Phase 1. For non-developer experts in the Ethics, UX, or Data Governance Guilds to effectively review a feature, they must be able to run and interact with it without a complex, error-prone manual setup. A simple, one-command setup makes their participation feasible. An ethicist with limited command-line experience is far more likely to provide the required holistic review if the setup process is git clone followed by docker-compose up. This task, therefore, is a direct technical enabler of the project's collaborative, interdisciplinary governance structure.
- **Implementation:** A docker-compose.yml file will be created in the repository's root directory. This file will define at least two services:
    1. app: This service will be built from a Dockerfile that defines the Python environment, installs dependencies from requirements.txt, and runs the FastAPI application.
    2. db: This service will use the official postgres:latest image directly from Docker Hub, fulfilling the database technology choice specified in Table 1.
- **Acceptance Criteria:** A new contributor, regardless of their technical background, must be able to clone the repository, run the single command docker-compose up, and have the entire application stack, including the database, running on their local machine and accessible via their browser.

2.4 Development and Validation of the "Hello, World" Test Plugin

This section focuses on validating the entire microkernel architecture with the simplest possible functional unit.

Task 2.4.1: Develop the Echo Plugin

- **Context:** This task serves a dual purpose. Tactically, it validates that the plugin API is functional and that the microkernel can correctly load and communicate with an external module. Strategically, it creates the *canonical reference implementation* for all future plugins. By providing a simple, well-documented, and working example, it dramatically reduces the cognitive load for new community contributors wanting to build their first "real" skill, thereby accelerating community-driven feature development.
- **Implementation:** A new directory, /plugins/echo, will be created. Inside this directory, a simple Python class will be developed. This EchoPlugin will implement the required interface to register with the microkernel and will have a single method: execute(input_text). This method will perform the trivial action of returning the input_text it received. The microkernel's startup logic will be configured to discover and load all plugins found in the /plugins directory.

Task 2.4.2: Write an Integration Test for the Echo Plugin

- **Implementation:** A new test will be added to the test suite. This will be an integration test, not a unit test. It will start the full application (using Docker Compose or a test-specific fixture), send an HTTP request to the /api/v1/invoke endpoint with a payload targeting the Echo plugin, and assert that the HTTP response body contains the same text that was sent in the request. This test proves that the end-to-end data flow—from API gateway, through the microkernel, to the plugin, and back—is working correctly.

2.5 Core API Documentation and Merge

The final step of Phase 2 is to document the work and integrate it into the main development line.

Task 2.5.1: Document the Plugin API

- **Context:** Clear, concise documentation is a cornerstone of a successful open-source project, as it empowers the community to build upon the core team's work.
- **Implementation:** A new documentation file, /docs/plugin_api.md, will be created. This document will explain the plugin architecture and provide a step-by-step guide on how to create a new plugin. It will cover the required class structure, the registration process, and the contract of the execute method. The fully implemented Echo plugin will be used as the primary, concrete example throughout the documentation.

Task 2.5.2: Submit and Review the Microkernel Pull Request

- **Implementation:** All the work completed during Phase 2, which will have been developed

on a dedicated feature branch (e.g., feature/microkernel-core), will be consolidated into a single, comprehensive pull request targeting the develop branch. This pull request will undergo the project's first formal peer review, setting the standard for all future code reviews. It will not be merged until all automated CI checks (linting and tests) have passed and it has received approval from the core maintainers.

Phase 3: Iterative Feature Development ("The Fit-Out")

This phase marks the transition from foundational architecture to the construction of user-facing capabilities. It operationalizes a core development loop, where each major feature is built as a self-contained, rigorously managed "sprint." The sprint workflow is itself a product of the project's principles, designed to ensure that every feature is developed, tested, and reviewed in a manner consistent with the "Architect's Oath." The sequence of sprints is strategically ordered to build foundational UI and data management elements before more complex application logic.

3.1 The Feature Sprint Workflow: A Repeatable Process

This section formally documents the end-to-end process that will be followed for every feature sprint in this phase and beyond. This repeatable workflow is the mechanism by which the project's abstract principles are translated into concrete development practices.

- **Description:** The workflow ensures that every feature is built with the same level of technical rigor and ethical oversight. Its most critical component is the "Awaiting Guild Review" state, which formalizes the interdisciplinary collaboration mandated by Specification 1.5. This process is designed to prevent the "siloed thinking" that leads to architectural and ethical failures by forcing technical decisions to be reviewed through multiple lenses—ethical, user experience, security, and data governance—*before* they are merged, not after. For example, a developer might implement a feature that is technically sound but has unforeseen ethical implications, such as a personalization feature that inadvertently creates a polarizing filter bubble. In a typical workflow, this might only be caught post-release. The Guild review process introduces a mandatory pause. The Pull Request is flagged for the Ethics Guild. An ethicist, using the "One-Click" environment from Phase 2, can easily run and test the feature's behavior, identify the risk of polarization (referencing the project's explicit commitment to combating it in Spec 5.1), and log this feedback directly in the PR. A discussion ensues, and the feature is redesigned *before* it ever reaches the main codebase, transforming the principles of the Constitution into a concrete, preventative quality gate.
- **Workflow Steps:**
    1. **Create Detailed Issue:** A new issue is created in the GitHub project board using the "Feature Request" template defined in Phase 1. The issue description must clearly define the "what" and "why" of the feature, including user stories and explicit acceptance criteria that reference the relevant sections of the technical specification (e.g., "Must fulfill all user data rights as defined in Table 3 of the specification").
    2. **Branch for the Feature:** A new feature branch is created from the latest develop branch (e.g., git checkout -b feature/personal-data-vault).
    3. **Develop and Test (TDD):** The application code and its corresponding unit and integration tests are written in tandem, following the TDD methodology established in Phase 2.

4. **Submit Pull Request & Trigger Reviews:** A pull request (PR) is opened to merge the feature branch into develop. The PR description must use a template that includes the mandatory "Guild Review Checklist." The author of the PR is responsible for checking the relevant boxes and formally requesting reviews from the designated Guild members using GitHub's "Reviewers" feature. This operationalizes the Guild structure from Specification 1.5.
5. **Pass Automated Checks & Guild Reviews:** The CI pipeline automatically runs all linting, testing, and other automated quality checks. The PR is blocked from merging until all checks pass AND formal approvals are received from all required peer reviewers and designated Guild reviewers.
6. **Merge and Deploy to Staging:** Once all human and automated approvals are granted, the code is merged into the develop branch. This merge will trigger a CI/CD workflow (to be configured as part of this phase) that automatically builds, containerizes, and deploys the develop branch to a public staging environment for final validation and community preview.

3.2 Sprint 1: The "Libre Order" Design System & Vitruvian UI Framework

This sprint builds the foundational visual and interactive language of the application.

- **Task 3.2.1: Implement Core Design Tokens**
  - **Context:** This task lays the groundwork for the entire visual system, directly implementing the requirements for a robust theming engine (Specification 7.2) and the visual language of "The Libre Order" design system (Specification 4.2).
  - **Implementation:** Define the project's foundational visual properties—colors, typography scale, spacing units, border radii—as a set of design tokens. These will be implemented as CSS custom properties to allow for easy, dynamic swapping, which is the core mechanism of the theming engine.
- **Task 3.2.2: Implement Core Component Library**
  - **Context:** This task builds the set of reusable, standardized UI elements defined in the "Vitruvian" UI Framework (Specification 3.3) and specified in Table 2 of "The Libre Order".
  - **Implementation:** Create the standardized, accessible (WCAG 2.1 AA compliant), and responsive components for: Primary Button, Input Field, Information Card, and Modal Dialog. Each component must be developed as a self-contained, testable unit with all states (e.g., default, hover, active, disabled, error) clearly defined and styled using the design tokens from the previous task.

3.3 Sprint 2: Main UI Layout and Onboarding Experience

This sprint constructs the main application shell and the user's critical first-run experience.

- **Task 3.3.1: Implement Main Tabbed UI Layout**
  - **Context:** This builds the primary application shell as detailed in Specification 7.4.
  - **Implementation:** Create the main application view containing the four navigation tabs in the specified order: Switchboard, Catalogue, Past Requests, and User Profile. The layout must be responsive, implementing the platform-specific designs: a bottom tab bar for mobile viewports and a vertical, right-side tab bar for desktop

viewports.

- **Task 3.3.2: Implement the Full Onboarding Flow**
  - **Context:** This implements the complete, step-by-step onboarding experience defined in Specification 3.4. This flow is critical as it introduces users to the core philosophical concepts of the assistant—modularity, user control, and privacy—from their very first interaction.
  - **Implementation:** Build the six-step onboarding process as a self-contained user flow. This includes the "Engine vs Car" metaphor screen, the Cloud vs. Local engine choice, the AI provider selection UI (for all seven specified providers), the initial plugin setup, the privacy and transparency explanation, and the final "All set!" screen that transitions the user to the main Switchboard.

3.4 Sprint 3: Switchboard and AI Provider Integration

This sprint builds the core interactive element of the assistant and connects it to its first "brain."

- **Task 3.4.1: Build the Switchboard UI**
  - **Context:** This task implements the detailed functionality of the Switchboard Tab as specified in Specification 7.4.A.
  - **Implementation:** Create the user interface for the Switchboard, including the always-visible request input field, the 12-slot plugin grid, and the thin activity line. Implement the front-end logic for the "light up" effect when plugins are active and the click-to-view-status functionality for each plugin slot.
- **Task 3.4.2: Implement Backend Logic for AI Provider Integration**
  - **Context:** This task represents the first functional connection to an external Large Language Model, making the assistant capable of intelligent responses.
  - **Implementation:** Create a backend service responsible for managing API keys, which will be stored securely after being entered by the user in the User Profile tab (Specification 7.4.D). Implement the logic to connect to at least one cloud-based AI provider (e.g., OpenAI) and one type of local AI provider (e.g., loading a GGUF-format model via a library like llama-cpp-python).
- **Task 3.4.3: Craft the Initial System Prompt**
  - **Context:** This task is a strategic, early application of the long-term research goal of Constitutional AI (Specification 12.1). Instead of treating AI alignment as a future problem, this task embeds the project's core values directly into the AI's behavior from its very first integration. The system prompt is the primary mechanism for controlling an LLM's behavior, tone, and constraints; it is, in effect, a mini-constitution. By proactively and carefully engineering this prompt, the project immediately implements a "CAI v0.1," ensuring the assistant's "personality" and ethical guardrails are aligned with the Architect's Oath.
  - **Implementation:** A detailed system prompt will be crafted and stored as a version-controlled text file in the repository for transparency and auditability. The prompt will be engineered based on established best practices to instruct the AI model to be "helpful, harmless, and honest". It will explicitly command the model to:
    - Adhere to the principles of the project's CONSTITUTION.md.
    - Refuse to process or store personally identifiable information.
    - Acknowledge and avoid generating biased or stereotyped content.
    - Be transparent about its identity as an AI and cite sources when providing factual information.

        - Refuse harmful or unethical requests, explaining the refusal in the context of its guiding principles.

3.5 Sprint 4: Plugin Catalogue

- **Task 3.5.1: Implement the Catalogue Tab UI**
    - **Context:** Implements the functionality of the Catalogue Tab as specified in Specification 7.4.B.
    - **Implementation:** Build the UI for the Catalogue tab, featuring a searchable and filterable list of all available plugins. Each entry will display the plugin's icon, name, and description. Implement the toggle switch functionality to enable or disable each plugin, which will control its availability on the Switchboard.

3.6 Sprint 5: Past Requests Tab

- **Task 3.6.1: Implement the Past Requests UI and Logic**
    - **Context:** Implements the user's interaction history view as specified in Specification 7.4.C.
    - **Implementation:** Build the list view for the Past Requests tab, displaying the timestamp, original request text, and icons of plugins used for each past interaction. Implement the backend logic to store this request history in the PostgreSQL database. Crucially, this includes storing a detailed log of parameters and outputs for each plugin involved in a request, which is necessary for the "View plugin usage log" feature.

3.7 Sprint 6: Personal Data Vault

This is a dedicated, high-priority sprint to implement the core privacy features of the application.

- **Task 3.7.1: Implement the Personal Data Vault UI and Backend**
    - **Context:** This sprint is dedicated to implementing Specification 6.2, the cornerstone of the project's commitment to user privacy, data autonomy, and compliance with GDPR principles.
    - **Implementation:** Create the dedicated UI section within the User Profile tab for the "Personal Data Vault." Implement all backend logic for securely storing user-specific data (such as learned preferences, request history, and any other PII) in an encrypted format within the PostgreSQL database.
- **Task 3.7.2: Implement GDPR User Rights Functionality**
    - **Implementation:** Implement the specific features that map directly to the user's fundamental data rights under GDPR, as explicitly defined in Table 3 of the technical specification. This is a non-negotiable compliance and trust-building feature set:
        - **Transparency (GDPR Art. 15, Right of Access):** Implement a clear, human-readable log that allows the user to view all personal data the assistant has stored about them.
        - **Control (GDPR Art. 16 & 17, Right to Rectification & Erasure):** Implement UI elements and backend logic that allow the user to edit and permanently delete any individual piece of data in the vault.
        - **Portability (GDPR Art. 20, Right to Data Portability):** Implement a

one-click "Export My Data" button that generates and downloads a complete, machine-readable JSON file of the user's entire vault.
        - **Consent (GDPR Art. 7, Consent):** Implement a granular consent manager UI where users can explicitly opt-in or opt-out of specific data processing activities.

3.8 Sprint 7: Transparency Dashboards

This sprint focuses on building features that provide radical transparency into the system's operation and composition.

- **Task 3.8.1: Implement the "Bill of Materials" Dashboard**
    - **Context:** This task fulfills Specification 2.3, which mandates a user-facing dashboard providing complete transparency into the system's components, pioneering a new standard of holistic system auditability.
    - **Implementation:** Create a new UI view that displays a clear, human-readable "Bill of Materials." The initial version will display data from a static configuration file, which will be updated as dependencies change. The dashboard will list:
        - All FOSS libraries and dependencies, their versions, and direct links to their licenses.
        - The AI models currently in use, with links to their model cards or research papers.
        - The training datasets used for core models, including their sources and licenses.
- **Task 3.8.2: Implement the "System Health" Dashboard**
    - **Context:** This task fulfills Specification 9.2, providing users with unprecedented, user-facing observability into the system's real-time performance and health.
    - **Implementation:** Create a "Diagnostics" or "System Health" panel in the settings. This UI will display simple, understandable visualizations for:
        - Key performance metrics (e.g., average response time).
        - The operational status of connections to external APIs (e.g., a green/red indicator for the AI provider connection).
        - A human-readable log of recent, non-critical errors (e.g., "Could not connect to weather service at 8:05 AM").

Phase 4: Community and Ecosystem ("The Open House")

This phase shifts focus from building core application features to implementing functionality that empowers the community, expands the assistant's utility into the public sphere, and formalizes the project for a public release and sustainable long-term growth. It is about opening the doors of the now well-constructed house to its inhabitants and connecting it to the wider digital commons.

4.1 Implementation of the Theming Engine

This section delivers on the promise of deep user customization, a key aspect of making the assistant a truly personal tool.

- **Task 4.1.1: Architect Front-End with Design Tokens**
  - **Context:** This task directly implements Specification 7.2, which mandates that the UI be built with theming as a core architectural requirement, not an afterthought. The use of design tokens is the specified mechanism for this implementation.
  - **Implementation:** The existing CSS codebase will be refactored to ensure that all visual properties (colors, fonts, spacing, etc.) are defined and consumed through the design token system established in Phase 3. The default first-party themes mandated by the specification will be created: a standard "Light" theme, a "Dark" theme, and a "High Contrast" theme designed for accessibility. A UI control will be added to the User Profile tab to allow users to switch between these themes.

4.2 Development of the Secure "Theme Marketplace"

This section builds the platform for community-driven aesthetic customization, a feature that requires a robust security architecture.

- **Task 4.2.1: Build the Theme Marketplace UI**
  - **Context:** This task implements the user-facing portion of the "Theme Marketplace" as described in Specification 7.3.
  - **Implementation:** A UI will be created within the application's settings for browsing, previewing, installing, and rating community-sourced themes. This interface will fetch a catalog of available themes from a public repository.
- **Task 4.2.2: Implement the Secure Theme Loading Architecture**
  - **Context:** This task addresses the critical "secure sandboxing mechanism" requirement from Specification 7.3. Allowing users to load arbitrary, community-generated CSS is a significant security risk, as CSS can be used to perform Cross-Site Scripting (XSS) attacks, exfiltrate data, or severely degrade the user experience. Therefore, a multi-layered, defense-in-depth security architecture is not optional; it is a direct enabler of the community-building goal. By demonstrating a robust and transparent approach to mitigating these risks, the project builds trust with security-conscious designers and users, encouraging them to participate in the ecosystem.
  - **Implementation:** The following three-layer security model will be implemented:
    1. **Server-Side Sanitization:** All user-submitted CSS will be processed by a server-side sanitization engine before being stored or served. This engine will operate on a strict whitelist of safe CSS properties and values. It will aggressively strip any potentially dangerous properties (e.g., behavior, expression), functions (e.g., url() with javascript: protocols), and malformed syntax.
    2. **Sandboxed Iframe Rendering:** The custom theme will not be applied to the main application DOM. Instead, the core content of the assistant will be rendered within a sandboxed <iframe>. The user's custom (and sanitized) CSS will be injected as a <style> tag only inside this iframe. The <iframe> element will have a sandbox attribute configured to disable all scripting, popups, form submissions, and top-level navigation (e.g., <iframe sandbox="allow-same-origin">), effectively isolating the themed content from the main application shell.
    3. **Strict Content Security Policy (CSP):** The server will deliver the page containing the sandboxed iframe with a strict Content Security Policy HTTP

header. The style-src directive of this policy will be configured to only allow stylesheets from 'self' (the application's own origin) and the specific SHA-256 hash of the sanitized, user-generated CSS that is being injected. This provides a final, powerful layer of defense, ensuring that even if the sanitizer or sandbox were somehow bypassed, the browser itself would refuse to execute any unauthorized inline styles or scripts.

- **Task 4.2.3: Integrate Marketplace with Repository via Pull Requests**
  - **Implementation:** A new, dedicated public GitHub repository (e.g., libre-assistant-themes) will be created. The CONTRIBUTING.md for this repository will document the process for submitting a new theme via a pull request. The CI pipeline for this repository will automatically run the CSS sanitizer and other validation checks on every PR. Once a theme is reviewed for quality and security and the PR is merged, an automated process will update the central theme catalog that the application consumes.

4.3 Development of "Public Square" Features

This section implements the features that fulfill the project's civic mandate, transforming the assistant from a personal utility into a tool for informed public engagement.

- **Task 4.3.1: Implement the "Civic Information Skill"**
  - **Context:** This task implements Specification 5.2, which calls for a skill that provides users with "easy, structured access to information about their elected officials" and elections. The implementation of this skill and the "Debate Summarizer" skill are not independent; they are two sides of the same coin fulfilling the project's broader civic mandate from Specification 5.1. The Civic Information Skill provides the foundational, factual data ("who represents me," "what is on the ballot"), while the Debate Summarizer provides a tool for critically analyzing the discourse *surrounding* that data. Together, they create a powerful civic technology toolkit that empowers users to be both informed and discerning.
  - **Implementation:** A new plugin will be developed that integrates with one or more reliable, public civic information APIs. Based on a review of available services, the **Google Civic Information API** stands out as a primary choice due to its comprehensive coverage of federal, state, and local officials, polling places, and ballot information, as well as its partnership with the Voting Information Project. The **ProPublica Congress API** will be considered as a secondary source for its exceptionally deep data on federal legislative activity, including bill details, votes, and committee work. The choice of APIs will be justified and documented via the following comparison table.

API	Data Coverage	Update Frequency	Key Strengths	Licensing/Usage Limits
Google Civic Info API	Federal, state, county, municipal officials; Polling places; Ballot info. US-only.	Dynamically updated through elections.	Comprehensive election data via Voting Information Project partnership. OCD-ID standard for interoperability.	Free up to 25,000 queries/day.

API	Data Coverage	Update Frequency	Key Strengths	Licensing/Usage Limits
ProPublica Congress API	Federal only (House & Senate). Members, votes, bills, committees, statements, lobbying.	Votes: every 30 mins. Bills: 6 times/day.	Deep legislative data (cosponsorships, amendments). Rich historical data back to ~1990s.	Free, requires API key. No specified query limit, but asks for responsible use.
OpenStates API	All 50 states, DC, PR. Legislators, bills, votes, committees.	Updates range from real-time via API to monthly for bulk data.	The most comprehensive source for *state-level* legislative data. Open-source data scrapers.	Free tier available, requires API key registration.

- **Task 4.3.2: Implement the "Debate Summarizer Skill"**
  - **Context:** This task implements the second "Public Square" feature from Specification 5.2: a tool designed to generate a "neutral, structured summary of the different arguments presented" in a piece of text, with the explicit goal of combating polarization and fostering critical thinking.
  - **Implementation:** A new plugin will be developed that accepts a URL as input. The plugin will scrape the content of the linked article (e.g., a news article, op-ed, or policy paper) and use an LLM to perform a specialized summarization task. The prompt for the LLM will be carefully engineered based on state-of-the-art NLP research in viewpoint and contrastive summarization. The prompt will instruct the model to execute a multi-step process:
    1. Identify the distinct viewpoints or perspectives present in the text (e.g., "arguments for the policy," "arguments against the policy").
    2. For each viewpoint, extract the key claims, arguments, and supporting evidence.
    3. Generate a separate, abstractive, and neutrally worded summary for each identified viewpoint.
    4. Present these summaries side-by-side or in a clearly delineated structure, rather than blending them into a single summary which could obscure important contrasts and nuances.

4.4 Formalization of the Automated Release Process

This section matures the project's CI/CD pipeline into a full-fledged, automated release machine.

- **Task 4.4.1: Enhance CI/CD Pipeline for Releases**
  - **Context:** This task refines the existing CI/CD pipeline from a development and testing tool into a formal release automation system, as described in Specification 10.3.
  - **Implementation:** A new workflow file, release.yml, will be added to the .github/workflows/ directory. This workflow will be configured to trigger only when a new version tag (e.g., v1.0.0, v1.0.1) is pushed to the main branch. Upon triggering,

the workflow will execute the following automated steps:
    - Build and containerize the production-ready application artifacts.
    - Automatically generate release notes by compiling commit messages since the last version tag.
    - Create a formal, public GitHub Release.
    - Upload the built artifacts (e.g., Docker images, executables) to the GitHub Release.

4.5 Incubation of Long-Term Research Initiatives

This final section lays the groundwork for the project's future evolution by making the long-term research goals visible and actionable for the community.

- **Task 4.5.1: Create Project Boards for Long-Term Research**
    - **Context:** This task directly implements the final step of the provided algorithm, making the ambitious future roadmap items from Part XII of the technical specification visible, transparent, and open to community participation in research and discussion.
    - **Implementation:** Dedicated GitHub Project boards will be created for the key long-term research initiatives:
        - **Roadmap Item 12.1: Constitutional AI Framework:** This board will be seeded with initial issues for discussion and research, such as: "Investigate RLAIF (Reinforcement Learning from AI Feedback) for Harmlessness Training" , "Design a Democratic Process for Community Constitution Creation" , and "Develop a Red-Teaming Framework for Testing Constitutional Alignment".
        - **Roadmap Item 12.2: Privacy-Preserving Personalization:** This board will be seeded with issues like: "Feasibility Study: Federated Learning for Preference Tuning," and "Research: Implementing Differential Privacy for Usage Analytics."
        - **Roadmap Item 12.3: Digital Guild Model:** This board will be seeded with issues focused on governance and community structure, such as: "Define Contribution Pathways: Apprenticeship to Mastery," "Specification for Asynchronous Guild Review Tooling," and "Formalize the Guild Charter and Responsibilities."

Works cited

1. Implementing Ethical Guidelines in Tech: A 10-Step Guide - Join The Collective, https://www.jointhecollective.com/article/from-policy-to-practice:-10-steps-for-implementing-ethical-guidelines-in-tech-organizations/ 2. Mailing List Code of Conduct - Open Source Initiative, https://opensource.org/codeofconduct 3. Your Code of Conduct | Open Source Guides, https://opensource.guide/code-of-conduct/ 4. Contributor Covenant - Organization for Ethical Source, https://ethicalsource.dev/projects/contributor-covenant/ 5. Contributor Covenant - Wikipedia, https://en.wikipedia.org/wiki/Contributor_Covenant 6. Contributor Covenant | A Code of Conduct for Digital Communities, https://www.contributor-covenant.org/ 7. Code of Conduct - Open Source Security Foundation, https://openssf.org/community/code-of-conduct/ 8. Contributor Covenant Code of Conduct - dplyr, https://dplyr.tidyverse.org/CODE_OF_CONDUCT 9. Open Source Guides | Learn how to launch

and grow your project., https://opensource.guide/ 10. The Rise of AI Ethics Boards: Implementing Accountability in Tech Companies - DM WebSoft, https://dmwebsoft.com/the-rise-of-ai-ethics-boards-implementing-accountability-in-tech-companies 11. How to Form an AI Ethics Board for Responsible AI Development - Shelf.io, https://shelf.io/blog/how-to-form-an-ai-ethics-board-for-responsible-ai-development/ 12. Constitutional AI | Tracking Anthropic's AI Revolution, https://www.constitutional.ai/ 13. Constitutional AI: Harmlessness from AI Feedback - Anthropic, https://www-cdn.anthropic.com/7512771452629584566b6303311496c262da1006/Anthropic_ConstitutionalAI_v2.pdf 14. What Is AI System Prompt Hardening? A Guide to Securing LLMs - Mend.io, https://www.mend.io/blog/what-is-ai-system-prompt-hardening/ 15. Mastering System Prompts for AI Agents | by Patric - Medium, https://pguso.medium.com/mastering-system-prompts-for-ai-agents-3492bf4a986b 16. What Is Prompt Engineering? | IBM, https://www.ibm.com/think/topics/prompt-engineering 17. Effective Prompts for AI: The Essentials - MIT Sloan Teaching & Learning Technologies, https://mitsloanedtech.mit.edu/ai/basics/effective-prompts/ 18. Best practices for prompt engineering with the OpenAI API, https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api 19. Highlights from the Claude 4 system prompt - Simon Willison's Weblog, https://simonwillison.net/2025/May/25/claude-4-system-prompt/ 20. Paper Review: Constitutional AI, Training LLMs using Principles - Medium, https://medium.com/mlearning-ai/paper-review-constituional-ai-training-llms-using-principles-16c68cfffaef 21. User Uploaded CSS safe? - Stack Overflow, https://stackoverflow.com/questions/6553788/user-uploaded-css-safe 22. How to prevent XSS attacks - Cloudflare, https://www.cloudflare.com/learning/security/how-to-prevent-xss-attacks/ 23. Safely insert external content into a page - Mozilla - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Safely_inserting_external_content_into_a_page 24. Cross Site Scripting Prevention - OWASP Cheat Sheet Series, https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html 25. Preventing Cross Site Scripting Vulnerabilities - Open edX Documentation, https://docs.openedx.org/en/latest/developers/references/developer_guide/preventing_xss/preventing_xss.html 26. How to prevent XSS (Cross Site Scripting) whilst allowing HTML input - Stack Overflow, https://stackoverflow.com/questions/7024419/how-to-prevent-xss-cross-site-scripting-whilst-allowing-html-input 27. Play safely in sandboxed IFrames | Articles - web.dev, https://web.dev/articles/sandboxed-iframes 28. <iframe>: The Inline Frame element - HTML - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/iframe 29. Content Security Policy (CSP) - HTTP - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP 30. CSP Allow Inline Styles - Content Security Policy, https://content-security-policy.com/examples/allow-inline-style/ 31. Content Security Policy Level 3 - W3C, https://www.w3.org/TR/CSP3/ 32. Content-Security-Policy: style-src directive - HTTP - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy/style-src 33. Civic Information API - Google for Developers, https://developers.google.com/civic-information 34. Voting Location Lookup Tools - The Voting Information Project, https://www.votinginfoproject.org/projects 35. Civic Information API: now connecting US users with their representatives, https://developers.googleblog.com/en/civic-information-api-now-connecting-us-users-with-their-r

epresentatives/ 36. ProPublica Congress API, https://projects.propublica.org/api-docs/congress-api/ 37. ProPublica Congress (Independent Publisher) - Connectors - Microsoft Learn, https://learn.microsoft.com/en-us/connectors/propublicacongressip/ 38. Contrastive Summarization of User Reviews: An Aspect-based Abstractive Approach - ACL Anthology, https://aclanthology.org/2024.paclic-1.48.pdf 39. Summarizing Contrastive Viewpoints in Opinionated Text - ACL Anthology, https://aclanthology.org/D10-1007.pdf 40. Summarization of Opinionated Political Documents with Varied Perspectives - arXiv, https://arxiv.org/html/2411.04093v2 41. (PDF) Summarising the points made in online political debates - ResearchGate, https://www.researchgate.net/publication/306094079_Summarising_the_points_made_in_online_political_debates 42. RLAIF: What is Reinforcement Learning From AI Feedback? - DataCamp, https://www.datacamp.com/blog/rlaif-reinforcement-learning-from-ai-feedback 43. Collective Constitutional AI: Aligning a Language Model with Public Input - Anthropic, https://www.anthropic.com/research/collective-constitutional-ai-aligning-a-language-model-with-public-input 44. Collective Constitutional AI: Aligning a Language Model with Public Input - arXiv, https://arxiv.org/html/2406.07814v1 45. LLM Red Teaming: The Complete Step-By-Step Guide To LLM Safety - Confident AI, https://www.confident-ai.com/blog/red-teaming-llms-a-step-by-step-guide 46. LLM red teaming guide (open source) - Promptfoo, https://www.promptfoo.dev/docs/red-team/ 47. Lessons From Red Teaming 100 Generative AI Products - arXiv, https://arxiv.org/html/2501.07238v1